# Concurrent Task Dispatcher Design Report

**Name:** Elian Pena
**Project:** Concurrent Task Dispatcher in Rust
**Course:** CSCI-3334-01-Spring 2026

This project is a Rust program that runs tasks with 8 workers. The program makes tasks, puts them in a queue, and then the workers take the tasks and run them. Each task must wait in the queue before a worker can pick it up. After the task is done, the program saves the results and uses them for the final numbers. I also compared two ways of choosing tasks. FIFO runs tasks in the order they were added, and optimized runs the shorter tasks first. I used this to see how each one changed the wait time, total run time, and worker usage.

---

## Architecture

The program has four main parts:

Task maker → Shared queue → Worker pool → Metrics

My program works like a simple set of steps, first a task is made, then it goes into the queue, then a worker takes it, runs it, and updates the final numbers. I used this setup because it is easier to understand and explain. The task maker creates 1000 tasks, and each task has a number, a type, a run time, and the time it was made. The task type is either CPU or IO, and the setup uses about 70% IO tasks and 30% CPU tasks, so not every task is the same. The queue is where tasks wait before they run, workers do not get tasks right away, they take them from the queue. This is also where the scheduling matters, FIFO keeps tasks in the same order, and optimized puts shorter tasks first.

The worker pool has 8 workers, so the program has a set number of threads instead of making a new thread for every task. Each worker checks the queue, takes a task, runs it, and updates the results. If the queue is empty, the worker stops. The metrics keep track of the final numbers, like completed tasks, wait time, total time, and worker usage, which helped me compare FIFO and optimized.

## Task Model

Each task in my program has a number, a type, a run time, and the time it was made, so the program can keep track of it. The type is either CPU or IO, and the run time tells the worker how long the task should run. The time it was made helps the program figure out how long the task waited before a worker started it.

I used CPU and IO tasks so the work would not all be the same. The setup uses about 70% IO tasks and 30% CPU tasks, like the amendment example. Some tasks also have different run times, so FIFO and optimized have something to compare.

This task information is used for the final numbers, like wait time and turnaround time. Wait time is how long the task waited before running, and turnaround time is how long it took from being made to being finished.

## Synchronization Strategy

The queue is shared by all 8 workers, so I had to make sure they do not mess it up when using it at the same time. I used Arc<Mutex<VecDeque<Task>>> for the queue because Arc lets the workers share it and Mutex makes only one worker use it at a time.

This matters because two workers should not be able to grab the same task. When a worker needs a task, it locks the queue, takes one task from the front, and then let's go of the lock, so the task can run without blocking the whole queue.

The final numbers are shared too because every worker updates them after finishing a task, so I used a Mutex there also. One downside is that workers might wait a little for the lock, but I only lock the queue when taking a task and only lock the numbers when updating them.

I did not use channels in my final design because all the workers needed to share one queue and update the same final numbers. I used shared state with Arc and Mutex instead, so the workers could safely use the same queue and metrics.

## Scheduling Policy

The program has two ways to pick tasks, FIFO and optimized. FIFO takes tasks in the same order they were added to the queue, so it is easy to understand. It is fair by order, but a short task might still get stuck behind a long task.

Optimized puts the shorter tasks first, so small tasks can finish faster. I used this to see if it would lower wait time. The downside is that longer tasks might wait more because the shorter tasks go first.

I used both modes because they are easy to compare, FIFO shows the basic way, and optimized shows what happens when the program tries to finish shorter tasks sooner. Optimized can help short tasks, but it can also be less fair for longer tasks because they can get pushed back.

## Metrics

The program prints the final numbers after all the tasks are done. It shows total runtime, makespan, tasks completed, average wait time, turnaround time, IO wait time, CPU wait time, max wait time, CPU usage, and how many workers were active.

Makespan means how long the whole program took. Wait time means how long a task waited before a worker started it, and turnaround time means how long it took from when the task was made to when it finished.

I added IO wait time and CPU wait time so I could see if one type of task waited more than the other. I also added CPU usage and active workers because it shows how busy the workers were during the run.

## Experiments

I ran two main experiments, one with FIFO and one with optimized. Both runs used 1000 tasks, about 70% IO and 30% CPU, 8 workers, and a 100% cap.

For FIFO, the tasks stayed in the same order they were added, so this showed how the basic scheduler worked.

For optimized, the shorter tasks went first, so this showed what happened when the program tried to finish smaller tasks sooner.

## Results:

FIFO simulation:
Total runtime: 4236 ms
Makespan: 4236 ms
Tasks completed: 1000 (CPU=300, IO=700)
Average wait time: 2087 ms
Average turnaround time: 2112 ms
Average wait IO only: 2087 ms
Average wait CPU only: 2089 ms
Max wait time: 3120 ms
Average CPU usage: 73.84%
Average workers active: 5.91 / 8

Optimized simulation:
Total runtime: 4246 ms
Makespan: 4246 ms
Tasks completed: 1000 (CPU=300, IO=700)
Average wait time: 1646 ms
Average turnaround time: 1672 ms
Average wait IO only: 1191 ms
Average wait CPU only: 2710 ms
Max wait time: 3864 ms
Average CPU usage: 73.66%
Average workers active: 5.89 / 8

The optimized run had a lower average wait time and lower average turnaround time, so it helped tasks finish sooner overall. The big change was with IO tasks, because their average wait time dropped from 2087 ms to 1191 ms. The downside is that CPU tasks waited longer in optimized mode, going from 2089 ms to 2710 ms, and the max wait time also went up. This shows the trade-off of shorter-tasks-first: it helps smaller tasks, but longer CPU tasks can get pushed back. The total runtime and CPU usage stayed almost the same for both runs, so the main difference was how the waiting time was split between IO and CPU tasks.

## Bug or Problem I Hit

One problem I had was making sure the workers stopped the right way. Since each worker runs in a loop, I had to make sure they did not keep running after all the tasks were done. I fixed this by checking the queue each time. If pop_front() gave back None, the worker stopped.

Another problem was the shared queue. Since all 8 workers use the same queue, I had to make sure two workers did not take the same task. I fixed this by using a Mutex, so only one worker could use the queue at once.

I also had a Rust warning early on because time_needed and created_at were not being used yet. Once I used those for running the task and tracking time, the warning went away.

## Trade-Offs and Conclusion

One trade-off was using one shared queue, because it made the program easier to follow, but sometimes a worker might have to wait if another worker is already using it. Another trade-off was FIFO compared to optimized, FIFO is simple because it keeps the same order, but a short task can get stuck behind a long task. Optimized helps shorter tasks finish faster, but longer tasks can wait more, so it can be unfair sometimes. Overall, this project helped me understand how a task dispatcher works in Rust. A task is made, put into a queue, picked up by a worker, run, and then added to the final results. It also showed me that the way tasks are picked can change wait time, total run time, and worker usage.

This was my first planning sketch when I had 4 workers and 500 tasks. The final version uses 8 workers and 1000 tasks, but the main flow stayed the same: tasks go into the queue, workers run them, and metrics are updated.